Numerical Method - Homework 1

B13902022 Yu-Chi,Lai

1

The cancellation issue arises when TRON computes the Conjugate Gradient (CG) step length α at the trust-region boundary. This is derived from the condition:

$$\|s + \alpha d\|^2 = \Delta^2 \tag{1}$$

Solving for the positive root of this quadratic equation yields:

$$\alpha = \frac{-s^\top d + \sqrt{(s^\top d)^2 + \|d\|^2(\Delta^2 - \|s\|^2)}}{\|d\|^2} \tag{2}$$

When $s^\top d > 0$, the term $\sqrt{(s^\top d)^2 + \ldots} - s^\top d$ involves the subtraction of two nearly equal floating-point numbers, leading to **catastrophic cancellation**. To ensure numerical robustness, LIBLINEAR (specifically in `tron.cpp`) utilizes the following piecewise implementation:

- **If $s^\top d \geq 0$:** The formula is rationalized to avoid subtraction:

$$\alpha = \frac{\Delta^2 - \|s\|^2}{s^\top d + \sqrt{(s^\top d)^2 + \|d\|^2(\Delta^2 - \|s\|^2)}} \tag{3}$$

- **Else ($s^\top d < 0$):** The standard quadratic formula is safe to use, as $-s^\top d$ is positive:

$$\alpha = \frac{\sqrt{(s^\top d)^2 + \|d\|^2(\Delta^2 - \|s\|^2)} - s^\top d}{\|d\|^2} \tag{4}$$

By switching between these algebraically equivalent forms based on the sign of $s^\top d$, the implementation maintains high precision even near the trust-region boundary.

2

I use the following code to reproduce the example ($f_i = 1, A = -64, B = 0$) given in the paper, and I got 0 using the first method (`calc1()`, $1 - p_i = 1 - \frac{1}{1+e^{Af_i+B}}$), while getting 1.60381e-28 when using the second method (`calc2()`, $1 - p_i = \frac{e^{Af_i+B}}{1+e^{Af_i+B}}$)

The second result is more precise, this is mainly because, when $(f_i, A, B) = (1, -64, 0)$, $\exp(Af_i + B) \approx 0$, it successfully avoid the subtraction between two close numbers in `calc1` (In this case, 1 and p_i), which leads to catastrophic cancellation.

```cpp
#include <bits/stdc++.h>
using namespace std;

double calc1(double f, double A, double B) {
    return 1.0 - 1.0 / (exp(A*f+B)+1.0);
}

double calc2(double f, double A, double B) {
    return exp(A*f+B) / (1.0+exp(A*f+B));
}

int main() {
    cout << calc1(1, -64, 0) << endl;
    // output: 0
    cout << calc2(1, -64, 0) << endl;
    // output: 1.60381e-28
}
```